

GETCERTS - Rexx Certificate Library for RACF® – Rexx Programmer Reference

June 4, 2026

GETCERTS is a library module that returns RACF certificate information to a Rexx program.

Table of Contents

Introduction	3
Overview of the Hypothetical Rexx Program	3
Loading and Calling GETCERTS.....	5
Initialization.....	5
Information Retrieval.....	6
Termination	6
Certificate and Ring Information Retrieval	6
Linkage.....	6
Return Codes and Associated Messages.....	6
Retrieving Error and Other Messages	7
Returned Certificate Information.....	7
Data Formats	8
GETCERTS Functions.....	8
Detailed Description of GETCERTS Functions	9
Common Parameters	9
CERTSUMM – Get Certificate Summary	10
RINGSUMM – Get Keyring Summary	12
CERTDETL – Get Certificate Detail	14
RINGCONN – Get Keyring Connected Certificates –	14
Trademarks	16
Sample Rexx Programs.....	17
Assumptions for all of the programs	17
Sample 1 – List all certificate expiration dates	18
Sample 2 – List all empty keyrings	19
Sample 3 – List all keyrings with no unexpired, trusted certificates	20

Introduction

GETCERTS is an X.509 certificate external environment (callable library) module for Rexx. GETCERTS is driven by ADDRESS commands from a Rexx program. In turn, GETCERTS inventories the RACF certificate database and populates Rexx stem variables with most of the details of the certificates in the RACF database. Similarly, GETCERTS will inventory the RACF keyrings and returns their details to the Rexx program. GETCERTS has no user interface beyond the ADDRESS command environment and a printable listing; it is intended that the invoking Rexx program would display or process the certificate information in some suitable manner.

Overview of the Hypothetical Rexx Program

The concept is an ISPF application that would provide an advanced X.509 certificate display on z/OS. In this paper I will refer to the ISPF application as “the Rexx program.”

In addition, we might make GETCERTS available to the z/OS community as a standalone CBT application. Users could develop their own certificate reports or other applications as they chose.

In the following discussion I will reference the sample Rexx EXEC, TGETCERT.

The envisioned ISPF application, the Rexx program, would start by installing GETCERTS as a Rexx “environment.” That process, and the basic calls to GETCERTS using Rexx ADDRESS, are documented below under **Loading and Calling GETCERTS**, and illustrated in the sample Rexx script, TGETCERT, the Rexx program would start by calling the CERTSUMM function, which is a prerequisite to any other functions.

The Rexx program would start by displaying a panel modeled after the SDSF job displays. Each detail line of the display would represent one certificate. The user could scroll up and down, and/or left and right. The fields to be displayed would be chosen at the discretion of the Rexx programmer, selected from the fields listed below under CERTSUMM. Some of the fields may be as wide as 67 characters, so the Rexx program would probably truncate them and/or allow user-variable column widths similar to what SDSF does. The display of some fields might be deferred, at the discretion of the implementer, to any Certificate Detail request.

A primary command would let the user alternatively display keyrings, one per detail line. To obtain the data for the display, the Rexx program would call the RINGSUMM function. RINGSUMM returns only five Rexx stem variables, so the panel design

would be somewhat simpler (although one of the fields, the keyring name, may be up to 237 characters wide, and may contain special characters).

An important concept is certificate and keyring “index numbers.” An index number is an integer, starting with 1, that is used as the tail of the various compound variables for the certificate or keyring. (Stemname.0 contains the count of tails starting from 1, in accordance with the typical Rexx convention.) The index number is also used by GETCERTS to identify other certificates or keyrings, such as in the CONCERT# keyring variable, which lists the index numbers of the certificates connected to the ring, separated by blanks. Thus WORDS(CONCERT#.tail) would give the number of certificates connected to the keyring with a keyring index of *tail*, and WORD(CONCERT#.tail, 1) would be the index number of the first certificate connected to the keyring. The sample Rexx script TGETCERT illustrates this logic. (All GETCERT returned variables with stem names ending in ‘#’ are similar lists of index numbers.)

It is envisioned that the certificate and keyring primary displays would support various “line commands” similar to the SDSF ‘S’ line command (subject to the discretion of the the Rexx program author). The anticipated commands, and a brief summary of how they might be implemented, are listed below.

Certificate Line Command Functions:

- List Detail. This would be implemented by performing the CERTDETL function, passing the index number of the certificate. I will leave it to the discretion of the implementor how to display the various returned data, including any data deferred from the primary certificate display.
- List Chain. This would be implemented by following the certificate chain in ISSUER# for each certificate. Note that a certificate *could* have multiple issuers. It is unlikely, but it is possible.¹ There is an example of how a List Chain would work in the sample Rexx EXEC, TGETCERT.
- List Dependents. This is the reverse of List Chain and is implemented by displaying the certificates listed in the certificate’s LEAF#. The sample script, TGETCERT, has examples.
- List Connected Keyrings. This function would require that a RINGSUMM function had been called. The list of connected keyrings is found in KEYRING#.

¹ <https://scotthelme.co.uk/cross-signing-alternate-trust-paths-how-they-work/>
<https://www.ssltrust.com.au/blog/understanding-certificate-cross-signing>

Keyring Line Command Functions

- List Connected Certificate Detail. This function would be implemented by calling the RINGCONN function and displaying the returned data. It would be good if the Certificate line commands were then available so one might do a List Detail, or List Chain, etc.

Loading and Calling GETCERTS

GETCERTS is invoked by a Rexx program as a Rexx “ADDRESS” environment. The Rexx program would install GETCERTS as a Rexx environment using the separate program EZSUBCOM.

EZSUBCOM must be available to the Rexx program. It may reside in ISPLLIB. GETCERTS must reside in the MVS content management chain (STEPLIB, etc.) or in the load library or load library concatenation identified by *loadlbdd* in the statements below. ISPLLIB is not searched for GETCERTS. EZSUBCOM is separately documented but the following is an overview of using EZSUBCOM to install GETCERTS as a Rexx environment.

EZSUBCOM must be invoked with Rexx CALL; it cannot be invoked with ISPF SELECT PGM.

Initialization

```
ADDRESS ISPEXEC "LIBDEF ISPLLIB DATASET ID(ezsubcom.LoadLib) STACK"  
ADDRESS TSO "ALLOC FI(loadlbdd) DA(getcerts.LoadLib) SHR"  
ADDRESS TSO 'ALLOC FI(GCPRINT) desired alloc operands'  
Call 'EZSUBCOM' 'ADDLESUB', 'GETCERTS', , , 'loadlbdd'
```

GCPRINT may be allocated to the terminal, to DUMMY or to a dataset as the the Rexx program author prefers. *It must be allocated before the call to EZSUBCOM ADDLESUB. If GETCERTS cannot open GCPRINT then the ADDLESUB call will fail.* GETCERTS opens GCPRINT RECFM=VBA, LRECL=133.

Rexx will load EZSUBCOM from ISPLLIB *but not GETCERTS*. If GETCERTS is not in the STEPLIB chain then you must allocate a DD to the appropriate load library and pass it as a parameter to EZSUBCOM as shown above.

CALL ‘EZSUBCOM’ returns its result in the Rexx special variable RESULT. RESULT is a string consisting of a return code 0, 4, 8, 12 or 16 following the usual MVS conventions and, if the return code is non-zero, a string describing the error or warning condition. Sample Rexx code:

```

Retvalue = RESULT
Parse var Retvalue retcode errmsg
If retcode <> 0 Then Do
  Say 'EZSUBCOM' retcode errmsg
  If retcode > 4 Then /* Deal with error */
End

```

Information Retrieval

See **Certificate and Ring Information Retrieval** and **Retrieving Error and Other Messages**.

Termination

```

CALL 'EZSUBCOM' 'DELETE', 'GETCERTS'
ADDRESS TSO 'FREE FI(GCPRINT)'
ADDRESS TSO 'FREE FI(LoadLbdd)'

```

See Initialization above for EZSUBCOM return result information.

Certificate and Ring Information Retrieval

Linkage

ADDRESS GETCERTS *function parameters*

The various functions and parameters are documented in detail below.

ADDRESS returns an integer return code in the Rexx special variable RC. If the CEEPIPI call returns a non-zero value (which in our experience is unlikely) then EZSUBCOM returns the CEEPIPI return code; otherwise EZSUBCOM returns the return code from the module. CEEPIPI subroutine invocation return codes are documented here: <https://www.ibm.com/docs/en/zos/3.2.0?topic=csis-return-codes> . All of the non-zero values are “should-not-occur” conditions.

Return Codes and Associated Messages

GETCERTS logs all error and warning conditions to the GCPRINT data set. GETCERTS returns :

- 0 – Successful function, no error or warning conditions detected
- 4 – One or more warning conditions detected.
- 8 – One or more error conditions detected *in the RACF certificate store*.
- 12 – One or more error conditions detected *in the parameters passed on ADDRESS or in executing them*.

- 16 – Fatal error condition in GETCERTS. GETCERTS could not continue. Contents of Rexx variables may not be valid.

In addition, for non-zero RC values, GETCERTS returns in the Rexx special variable RESULT a printable error message for the first detected condition that caused the error of that severity. In other words, if GETCERTS returns a 12 in RC then RESULT will contain an error message corresponding to the condition that caused the severity 12 error.

Retrieving Error and Other Messages

GETCERTS can also return error and other messages in Rexx compound variables, GC_MSGS.*n*, where 'n' is an integer following the usual Rexx convention, with a count in GC_MSGS.0. To control the return of messages, specify MSGLEVEL *nn*|NO on any ADDRESS command. See Common Parameters below. The GC_MSGS stem is always GC_MSGS; it is *not* affected by any PREFIX specification.

The GC_MSGS stem is populated “cumulatively” by each ADDRESS command. If your first ADDRESS command returns GC_MSGS.1 and GC_MSGS.2, then any messages returned by the next ADDRESS command would start from GC_MSGS.3. If you want to process only the messages from one ADDRESS command then you should save the value of GC_MSGS.0 before that ADDRESS command and only process GC_MSGS tails starting from that value plus one.

Most certificate-related error messages, if any, tend to be generated on a CERTSUMM call. Use caution if you specify both VERBOSE and MSGLEVEL 0: GETCERTS may generate thousands of informational VERBOSE messages.

GETCERTS messages follow the usual IBM and z/OS message conventions. The following are examples of an informational, warning, error and severe error message respectively:

```
CMC0058I GETCERTS() subroutine entered with ADDRESS GETCERTS 'CERTSUMM PREFIX C_'
CMC0011W The following certificates share a common Subject Key ID, which may
cause validation problems. Key ID: 35333A37393A42463A35413A41413A32423A34413A43
CMC0048E Should not occur: ring JES2UID/FTP.Key.Ring contains unknown certificate
SYS06 'CMC Endpoint'
CMC0041S The command token PERFIX is not recognized
```

Returned Certificate Information

Certificate and keyring data is returned to the Rexx program in Rexx variables. The Rexx program may specify name prefixes – a character string to be prepended to the defined variable names.

Data Formats

Date and Time Format. Dates and times are returned in ISO 8601 (2026-02-21T11:46:33): it's a standard, it's sortable, it's unambiguous for international customers, and it's easy to truncate to 10 characters if time is not needed. Certificate timestamps are formatted as UTC.

Hexadecimal Format. Unpunctuated uppercase hex is common on the mainframe: 08C6A634815E296535A4... Uppercase hex with colon as a byte separator is used by RACF: 7A:2B:6A:F3:C4:91:DF:D3... in some places and unpunctuated lowercase hex in others. Lowercase hex with colon as a byte separator is used by OpenSSL: 08:c6:a6:34:81:5e:29:65:21:35:a4... Gskkyman uses uppercase hex with blank as a byte separator in some places and unpunctuated lowercase hex in others. GETCERTS follows RACF's lead and uses 7A:2B:6A:F3:C4:91:DF:D3... but I am open to suggestions.

True/False Format. TRUE and FALSE are returned as 1 and 0 for convenient usage from Rexx. The Rexx program might choose to display the values as TRUE/FALSE, YES/NO, TRUST/NOTRUST or otherwise as the programmer chose.

GETCERTS Functions

the Rexx program will call the **CERTSUMM** (Get Certificate Summary) and **RINGSUMM** (Get Ring Summary) functions once and display summary information. Then, if the user asks for detail for a particular certificate, the Rexx program will call **CERTDETL** (Get Certificate Detail) for that certificate. Or if a user asks for the connected certificates for a particular keyring then the Rexx program would call **RINGCONN** (Get Keyring Connected Certificates).

If the user wished to refresh the certificate or ring data it should be possible to call the CERTSUMM and/or RINGSUMM functions again, but this has not been tested.

Summary information is returned in a set of Rexx compound variables with a defined stem name and a numerical tail.

For example, assuming a certificate variable prefix of C_, the label information for the certificates will be returned in C_LABEL.1, C_LABEL.2, C_LABEL.3 and so forth, with a count of certificates in C_LABEL.0. Similarly, the "not after" date for the certificates will be returned in C_NOTAFTER.1, C_NOTAFTER.2, C_NOTAFTER.3, and so forth.

In this paper I will refer to the Rexx stem variable tail value (1, 2 and 3 in the examples above) as the certificate's or keyring's "index number."

The detail information would similarly be returned in a set of Rexx variables, but the detail variables would for the most part be simple Rexx variables. The certificate detail data would apply to only the one certificate so there would be no need for compound qualification. The exception would be several pieces of detail information that might repeat a variable number of times. That information would be returned in compound variables using the usual .1, .2, .3 and .0 convention.

Why not simply return all of the certificate's or keyring's data in the summary variables? The main reason is that some of the detail data repeats in stem variables with numeric tails. As the summary data already repeats in stem variables with numeric tails, that would imply "double numeric tails" – variables like C_SAN.37.3 for the third SAN (Subject Alternate Name) for certificate number 37. I feel that would be confusing.

Why a separate RINGINFO call? Because ring retrieval requires additional permissions beyond those required for certificate retrieval, and some users might not have those permissions and would prefer not to get security violations.

Detailed Description of GETCERTS Functions

Common Parameters

The common parameters are

[MSGLEVEL *nn*|NO] [PREFIX *prefix*] [VERBOSE]

These parameters may be specified with any of the functions.

MSGLEVEL *nn*|NO specifies the severity level of messages to be returned in the GC_MSGS compound variable. Specify where an integer or N.; NO indicates that no messages are to be returned. *nn* is a severity level: if you specify 0 then all messages generated by GETCERTS are returned; if you specify 4 then warning and error messages are returned but not informational messages; if you specify 8 then only error messages are returned; and if you specify 12 then only severe error messages are returned. Severe error messages always diagnose programming or similar errors such as a mis-spelled command token. If you omit MSGLEVEL then it defaults to MSGLEVEL 8. See **Retrieving Error and Other Messages** above.

PREFIX *prefix* specifies an optional prefix to be prepended to the returned Rexx variable names. For example, assuming a certificate variable prefix of C_, the label information for the certificates will be returned in C_LABEL.1, C_LABEL.2, C_LABEL.3 and so forth, with a count of certificates in C_LABEL.0. The prefix may be

any length but prefixes longer than four characters may have a slight negative effect on performance.

VERBOSE specifies that additional informational messages are to be output to GCPRINT by GETCERTS. The VERBOSE output is potentially voluminous.

CERTSUMM – Get Certificate Summary

CERTSUMM retrieves certificate summary information for all certificates.

ADDRESS GETCERTS CERTSUMM [common parameters]

CERTSUMM is required as the first parameter. It specifies the certificate summary function.

See also Common Parameters above.

The remaining parameters are optional and may be specified in any order.

The CERTSUMM function requires the following RACF permissions:

- READ authority to IRR.RADMIN.RLIST (or IRR.RADMIN.*) in the FACILITY class.
- READ authority to CSFOWH in the CSFSERV class; or
CSF.CSFSERV.AUTH.CSFOWH.DISABLE resource profile defined in the XFACILIT class.

And either

- If the *.IRR_VIRTUAL_KEYRING.LST profile or any
userid.IRR_VIRTUAL_KEYRING.LST profile exists in the RDATA LIB class, then
READ authority to the *.IRR_VIRTUAL_KEYRING.LST profile.
- Otherwise UPDATE authority to IRR.DIGTCERT.LISTRING in the FACILITY class.

The CERTSUMM function may be invoked as many times as the Rexx program wishes. It must be invoked before any other function.

Rexx variables set by CERTSUMM are listed in the table below. The Rexx stem names are “prefixed” as described above. The Rexx program might choose to defer some of these fields to any detail display.

Rexx Stem	Description	Notes
AUTHKID	Authority Key Identifier	See <u>Hexadecimal Format</u> above.
CAUSAGE	From basic constraints	1 if CA:TRUE; 0 if CA: FALSE

Rexx Stem	Description	Notes
CERTDADD	Date certificate added to RACF	See <u>Date and Time Format</u> above. No time; only date. Time is recorded as 00:00:00.
CERTUADD	RACF ID of user who actually added certificate to RACF	1 to 8 characters. Never irrcerta or irrsitec. Not necessarily the same as the certificate's CERTUSER.
CERTUSER	Certificate owner	1 to 8 characters. CERTAUTH and SITE returned as irrcerta and irrsitec (lower case) respectively.
FINGERPR	SHA-256 fingerprint of the certificate	32 bytes, reported as 63 characters, as reported by RACF. See <u>Hexadecimal Format</u> above.
ISSUER	Full issuer name	May be up to 2048 characters
ISSUER#	Index number(s) of issuer certificates	Normally at most one issuer, but because of cross-signing, expired intermediates and other somewhat unusual circumstances a certificate may have multiple issuers. (See footnotes under Overview of the Hypothetical Rexx Program .) Multiple Issuer index numbers are separated by blanks. Empty string if no issuer certificate present. <i>Empty string for self-signed certificates.</i>
ISSUER\$	Short issuer name	Issuer name shortened to most significant RDN by algorithm. Up to 67 characters, but the Rexx program may display less if so chooses.
KEYALGO	Public key algorithm	Name such as rsaEncryption
KEYLEN	Private key length in bits	
KEYTYPE	Private key type	Key type and location, such as "RSA (in RACF)"
LABEL	Certificate label	1 to 32 characters. May include embedded special characters, including blanks and quotes. Quotes are not doubled by GETCERTS.
LEAF#	Index numbers of certificates signed by this certificate	Empty string if none. Multiple index numbers separated by blanks. "Self" not included for self-signed certificates.

Rexx Stem	Description	Notes
NOTAFTER	Certificate expiration date and time	See Date and Time Format above.
NOTBEFOR	Certificate start date and time	See Date and Time Format above.
PRIVKEY	Is there a private key	1 if private key in RACF, 0 if not. ²
SELSIGN	Certificate is self-signed	1 if self-signed; 0 if not
SERIAL	Certificate serial number	See <u>Hexadecimal Format</u> above.
SIGALGO	Signature algorithm	A name like sha256WithRSAEncryption
SUBJECT	Full subject name	May be up to 2048 characters
SUBJECT\$	Short subject name	Subject name shortened to most significant RDN by algorithm. Up to 67 characters, but the Rexx program may display less if so chooses.
SUBJKID	Subject Key identifier	See <u>Hexadecimal Format</u> above.
THUMBPR	SHA-1 Thumbprint	20 bytes, formatted as 39 characters (used by Windows). See <u>Hexadecimal Format</u> above.
TRUST	Certificate trust status	1 if trusted; 0 if not

RINGSUMM – Get Keyring Summary

RINGSUMM retrieves keyring summary information below for all rings and certificate ring connections. May not precede the CERTSUMM call.

ADDRESS GETCERTS RINGSUMM [common parameters]

RINGSUMM is required as the first parameter. It specifies the keyring summary function.

See also Common Parameters above.

² Meaningful only if certain RACF facilities are permitted; otherwise will return 0. See Usage note 4 in <https://www.ibm.com/docs/en/zos/3.2.0?topic=library-usage-notes>

The remaining parameters are optional and may be specified in any order.

RINGSUMM requires the following additional RACF permissions:

- UPDATE authority to IRR.DIGTCERT.LISTRING in the FACILITY class.
- If any keyring profiles exist in the RDATA LIB class, then READ authority to **LST in RDATA LIB.

The RINGSUMM command may not be invoked before CERTSUMM, and may only be invoked once after each CERTSUMM without an intervening CERTSUMM invocation.

Rexx variables set by RINGSUMM are listed in the tables below. The Rexx program might choose to defer displaying some of these fields to any certificate detail function.

Additional certificate variables, indexed by certificate “index number”:

Rexx Stem	Description	Notes
KEYRING#	Keyring index numbers of connected keyrings	Integers separated by blanks. Empty string if none or if Get Ring Summary has not been called. Sorted into ascending keyring index sequence.

Keyring variables, indexed by a keyring “index number” as a tail; for example, RINGUSER.9. The .0 tails contain a count of elements.

Rexx Stem	Description	Notes
CONCERT#	Certificate index numbers of connected certificates.	Zero or more integers separated by blanks, sorted into ascending sequence. An empty string if no connected certificates. The Rexx program may access the userid, label and so forth of the certificates as desired using the returned certificate index number and the appropriate Rexx variables.
NAME	Keyring name	1 to 237 characters. May contain lowercase and special characters.
RINGDADD	Date keyring added to RACF	See <u>Date and Time Format</u> above. No time; only date. Time is recorded as 00:00:00.
RINGUADD	RACF ID of user who actually added keyring to RACF	1 to 8 characters. Never irrcerta or irrsitec. Not necessarily the same as the Keyring’s RINGUSER.
RINGUSER	Keyring owner	1 to 8 characters. Never irrcerta or irrsitec.

CERTDETL – Get Certificate Detail

CERTDETL retrieves certificate detail information for one certificate, specified by certificate index number. The function may be called multiple times to retrieve information for multiple certificates.

ADDRESS GETCERTS CERTDETL CERTINDX *index* [common parameters]

CERTDETL is required as the first parameter. It specifies the certificate detail function.

CERTINDX *index* is required and specifies the certificate index number of the certificate for which detail information is to be returned.

See also Common Parameters above.

The remaining parameters are optional and may be specified in any order.

CERTDETL may only be invoked after CERTSUMM.

The certificate detail variables are stem variables following the usual Rexx convention. Assuming no change in prefix, each call to the **CERTDETL** function overwrites the variables from the previous call.

Rexx Variable	Description	Notes
EXTUSAGE	Extended key usage	Stem variable. May be multiple usages. Character strings such as "Server Authentication", "Client Authentication", "Code Signing", etc.
KEYUSAGE	Key usage	Stem variable. May be multiple usages. Character strings such as "Digital Signature", "Content Commitment", "Key Encipherment", etc.
SAN	Subject Alternative Names	Stem variable; may be dozens or more. Format DNS: www.foobar.com

RINGCONN – Get Keyring Connected Certificates –

RINGCONN retrieves a list of connected certificates and their attributes for one keyring, specified by keyring index number. The function may be called multiple times to retrieve information for multiple keyrings.

ADDRESS GETCERTS RINGCONN RINGINDX *index* [common parameters]

RINGCONN is required as the first parameter. It specifies the ring connections function.

RINGINDX *index* is required and specifies the keyring index number of the keyring for which connection information is to be returned.

See also Common Parameters above.

The remaining parameters are optional and may be specified in any order.

RINGCONN requires the following RACF permissions:

- READ authority to <virtual ring owner>.IRR_VIRTUAL_KEYRING.LST in the FACILITY class.

RINGCONN may only be invoked after RINGSUMM and before invoking another CERTSUMM. Any number of CERTDETL and/or RINGCONN commands may precede a RINGCONN.

A list of connected certificates and their attributes are returned as Rexx compound variables with a numeric tail, following the usual conventions. Each tail number represents the attributes of one connected certificate. A keyring may have no connected certificates, in which case all of the listed Rexx Stems will have a tail .0 with a value of 0. Assuming no change in prefix, each call to the **RINGCONN** function overwrites the variables from the previous call.

Rexx Stem	Description	Notes
CERTIF#	Certificate index number	The Rexx program may access the userid, label and so forth of the certificate as desired using the returned certificate index number and the appropriate Rexx variables.
DEFAULT	Whether certificate is the default on the keyring	1 = default; 0 = not the default. Note that a keyring has at most one and may have no default certificate.
USAGE	Certificate usage on keyring	CERTAUTH, PERSONAL or SITE ³

³ Two other values are possible. <https://www.ibm.com/docs/en/zos/3.2.0?topic=parameters-function-specific-parameter-list-datagetfirst-datagetnext> documents X'FFFFFFF5' as "Reserved." GETCERTS will return "RESERVED" for this usage value. For values other than the above, GETCERTS will return the value in hex as X'xxxxxxx'.

Trademarks

IBM®, RACF® and z/OS® are registered trademarks of International Business Machines Corporation. UNIX® is a registered trademark of The Open Group. No association is implied.

* * *

Sample Rexx Programs

Assumptions for all of the programs

- EZSUBADR (alias EZSUBCOM) and GETCERTS are in the program object (load) library named in the statement
LOADLIB = 'MY.LOAD'
- Running under TSO or TSO batch
- User has RACF permissions as indicated for each function

Sample 1 – List all certificate expiration dates

Lists all certificates with their expiration dates.

```
/* REXX program to demonstrate GETCERTS by showing all expiration dates */
```

```
Say "REXX program to demonstrate GETCERTS by showing all expiration dates"
```

```
LOADLIB = 'MY.LOAD'
```

```
If Sysvar("SYSISPF") = "ACTIVE" Then Do
  ADDRESS ISPEXEC "LIBDEF ISPLLIB DATASET ID('LOADLIB') STACK"
End /* SYSISPF */
```

```
ADDRESS TSO "ALLOC FI(GCPRINT) SYSOUT"
ADDRESS TSO "ALLOC FI(MYLOADDD) DA('LOADLIB') SHR"
```

```
/* Use EZSUBCOM to load GETCERTS */
Call 'EZSUBCOM' 'ADDLESUB', 'GETCERTS',,, 'MYLOADDD'
ReturnCode = RC
If ReturnCode < 0 | ReturnCode > 4 Then Do
  Say "Return Code" ReturnCode "from EZSUBCOM ADDLESUB GETCERTS"
  Signal Sig_EndPgm
End /* Bad RC from EZSUBCOM */
```

```
/* Get basic certificate information */
ADDRESS GETCERTS 'CERTSUMM'
ReturnCode = RC
If ReturnCode > 0 Then Say "RESULT =" RESULT
If ReturnCode > 8 Then Signal Sig_EndPgm
```

```
/* For each certificate say the expiration date */
Do i = 1 to CERTUSER.0
  Say NOTAFTER.i CERTUSER.i LABEL.i SUBJECT$.i
End i
```

```
/* Unload GETCERTS */
Call 'EZSUBCOM' 'DELETE', 'GETCERTS'
```

```
/* End of REXX EXEC */
Sig_EndPgm:
ADDRESS TSO "FREE FI(GCPRINT)"
ADDRESS TSO "FREE FI(MYLOADDD)"
If Sysvar("SYSISPF") = "ACTIVE" Then Do
  ADDRESS ISPEXEC "LIBDEF ISPLLIB"
End /* SYSISPF */
Exit 0
```

Sample 2 – List all empty keyrings

Lists all keyrings that contain no certificates.

```
/* REXX program to demonstrate GETCERTS by showing all empty keyrings */

Say "REXX program to demonstrate GETCERTS by showing all empty keyrings"

LOADLIB = 'MY.LOAD'

If Sysvar("SYSISPF") = "ACTIVE" Then Do
  ADDRESS ISPEXEC "LIBDEF ISPLLIB DATASET ID('LOADLIB') STACK"
  End
/* SYSISPF */

ADDRESS TSO "ALLOC FI(GCPRINT) SYSOUT"
ADDRESS TSO "ALLOC FI(MYLOADDD) DA('LOADLIB') SHR"

/* Use EZSUBCOM to load GETCERTS */
Call 'EZSUBCOM' 'ADDLESUB', 'GETCERTS',,, 'MYLOADDD'
ReturnCode = RC
If ReturnCode < 0 | ReturnCode > 4 Then Do
  Say "Return Code" ReturnCode "from EZSUBCOM ADDLESUB GETCERTS"
  Signal Sig_EndPgm
End /* Bad RC from EZSUBCOM */

/* Get basic certificate information */
ADDRESS GETCERTS 'CERTSUMM'
ReturnCode = RC
If ReturnCode > 0 Then Say "RESULT =" RESULT
If ReturnCode > 8 Then Signal Sig_EndPgm

/* Get keyring information */
ADDRESS GETCERTS 'RINGSUMM'
ReturnCode = RC
If ReturnCode > 0 Then Say "RESULT =" RESULT
If ReturnCode > 8 Then Signal Sig_EndPgm

/* List each empty keyring */
Do i = 1 to NAME.0
  If CONCERT#.i = '' Then Say RINGUSER.i NAME.i
End i

/* Unload GETCERTS */
Call 'EZSUBCOM' 'DELETE', 'GETCERTS'

/* End of REXX EXEC */
Sig_EndPgm:
ADDRESS TSO "FREE FI(GCPRINT)"
ADDRESS TSO "FREE FI(MYLOADDD)"
If Sysvar("SYSISPF") = "ACTIVE" Then Do
  ADDRESS ISPEXEC "LIBDEF ISPLLIB"
  End
/* SYSISPF */
Exit 0
```

Sample 3 – List all keyrings with no unexpired, trusted certificates

Lists all keyrings that contain no unexpired, trusted certificates. Lists the owner, the first 20 characters of the ring name and the userid that defined the keyring, with the date that they defined it. Sample output:

Owner	Name	Who	Added and When
ABSMITH	Smith.Ring	FRIKDJ	2026-01-25
FRED	fred.testring	FRED	2025-07-03
IBMCERT	IBMInternetServiceRe	IBUSER1	2010-09-21
TN3270	TN3270RING	IBUSER1	2011-09-06

```
/* REXX program to demonstrate GETCERTS by showing unusable keyrings */

Say "Rexx program to demonstrate GETCERTS by showing all keyrings with no"
Say "unexpired certificates"

LOADLIB = 'MY.LOAD'

If Sysvar("SYSISPF") = "ACTIVE" Then Do
  ADDRESS ISPEXEC "LIBDEF ISPLLIB DATASET ID('LOADLIB') STACK"
  End
/* SYSISPF */

ADDRESS TSO "ALLOC FI(GCPRINT) SYSOUT"
ADDRESS TSO "ALLOC FI(MYLOADDD) DA('LOADLIB') SHR"

/* Set up a comparison time in ISO 8601 format 2025-12-19T00:00:00 */
DateNow = DATE('S'); TimeNow = TIME('N')
Now = Left(DateNow,4) || "-" || Substr(DateNow,5,2) || "-" || Right(DateNow,2)
Now = Now || "T" || TimeNow

/* Use EZSUBCOM to load GETCERTS */
Call 'EZSUBCOM' 'ADDLESUB', 'GETCERTS',,, 'MYLOADDD'
ReturnCode = RC
If ReturnCode < 0 | ReturnCode > 4 Then Do
  Say "Return Code" ReturnCode "from EZSUBCOM ADDLESUB GETCERTS"
  Signal Sig_EndPgm
End /* Bad RC from EZSUBCOM */

/* Get basic certificate information */
ADDRESS GETCERTS 'CERTSUMM'
ReturnCode = RC
If ReturnCode > 0 Then Say "RESULT =" RESULT
If ReturnCode > 8 Then Signal Sig_EndPgm

/* Get keyring information */
ADDRESS GETCERTS 'RINGSUMM'
ReturnCode = RC
If ReturnCode > 0 Then Say "RESULT =" RESULT
If ReturnCode > 8 Then Signal Sig_EndPgm
```

```

/* For each keyring, check expiration & trust of each connected certificate */
Say "Owner      Name                               Who Added and When"
Say "-----"
Do k = 1 to NAME.0
  IsUseful = 0
  Do w = 1 to Words(CONCERT#.k)
    c = Word(CONCERT#.k, w)
    If NOTAFTER.c > Now Then Do
      IsUseful = 1
      Leave
    End
  End w
  If \ IsUseful Then Do
    Say Left(RINGUSER.k,8) Left(NAME.k,20) Left(RINGUADD.k,8) Left(RINGDADD.k,10)
  End
End k

/* Unload GETCERTS */
Call 'EZSUBCOM' 'DELETE', 'GETCERTS'

/* End of Rexx EXEC */
Sig_EndPgm:
ADDRESS TSO "FREE FI(GCPRINT)"
ADDRESS TSO "FREE FI(MYLOADDD)"
If Sysvar("SYSISPF") = "ACTIVE" Then Do
  ADDRESS ISPEXEC "LIBDEF ISPLLIB"
End
Exit 0

```